Liber Primus: where things stand

A plain-English overview, the Phase 10 result, and what an external review changed · 27 October 2026

Risk Averse Technology Company · repository github.com/RiskAverseTech/cicada-ioc at commit f58489d

---

PART 1 · THE PLAIN-ENGLISH VERSION

What Liber Primus is

Cicada 3301 was an anonymous group that ran three public puzzle hunts, in 2012, 2013 and 2014. Each started with an image posted online and led solvers through cryptography, steganography, literature and physical clues across several countries. Nobody has ever confirmed who they were or what the hunts were for. The 2014 hunt ended at a book called Liber Primus, "First Book." It is 58 pages of text written in an old Germanic rune alphabet, with each of the 29 runes also assigned a prime number, a table Cicada called the Gematria Primus. A handful of pages were solved quickly in 2014 and 2015. They turned out to be short philosophical texts in English, enciphered with simple methods: a reversed alphabet, a repeating keyword like DIVINITY, a stream built from prime numbers. The remaining pages, about 55 of them holding roughly 13,000 runes, have resisted every attempt since. Cicada's last authenticated message, in 2016 as far as the record shows, said in effect that Liber Primus is the way and nothing else is genuine. Then silence. So the unsolved pages are the last thread of a twelve-year-old mystery.

Why it is hard, in one idea

If you count how often each rune appears on the unsolved pages, every rune appears equally often. If you count pairs, pairs are flat too. By almost every measure the text is indistinguishable from random noise, which is what a good cipher produces. There is one exception. A rune almost never follows itself. Random text would double up about 445 times in 13,000 runes. The pages do it 86 times. That fivefold shortage was noticed by the community around 2014 and it is the only crack anyone has found. The cleanest explanation is that whatever generated the key looked at the previous rune and avoided repeating it, about four times out of five. But that explanation also says the key is independent of anything you can read, which means it is a one-time pad: unbreakable without the key itself.

The state of play in October 2026

The solving community lives on a Discord server and a wiki. Over the past year a new kind of participant appeared: people driving AI coding agents through enormous systematic sweeps. At least five such projects published results in 2026, each with a ledger of tested-and-failed hypotheses, and all of them converged on the same doublet shortage as the only structure. None of them has read the others closely, so the same ideas get retested. The human community has largely moved on to the hope that the key lives somewhere outside the book.

Where our repo fits

Ours is a sixth such project, started on October 23 from a Quorum debate. We did not set out to solve it. We set out to run each idea the way an experiment should be run: state the test before looking, plant a fake solution first to prove the test can see one, then run it on the real pages and publish the failure as a failure. We also built the ledger that compares the other five projects, so that a new idea can be checked against

everyone's work in minutes. Our contribution so far is nine phases of clean negative results, several of them closing lanes nobody else had closed, and one genuinely new finding from yesterday: the shortage is exactly the same on every page, which rules out a mixture of methods.

What is running right now

Yesterday's Quorum panel said three remaining explanations for the 81% figure could not be told apart. We showed by simulation that two of them can, and that only two mechanisms fit every number: the independent key with an anti-repeat habit, which is unbreakable, or a strict never-repeat key with about one meaningless filler rune in five slipped in at unknown places. That second one is the last version that could still be breakable, because the filler runes would be hiding a key drawn from the book's own words and number streams, which we already know how to test. The four shards running now try exactly that: every book-derived key, both ways of reading the F rune, with the decoder allowed to call any rune a filler. We planted fake pages first and the decoder found 15 of 18 of them.

What happens after the shards

If a real page lights up, we inspect it before believing it, since the pre-registered bar is high for a reason. If nothing does, which is the likely outcome, then the breakable branch of the last remaining explanation is closed, and the honest statement becomes: the unsolved pages are consistent only with key material that was never published. We write that up as section 22, offer the finding to the other ledgers, and the repo's value becomes the method and the tooling, plus the hypothesis loop where a model panel proposes and the statistics kill. The broader plan is for that loop to be the product: Quorum generates the next idea, the repo tests it in minutes, and the whole exchange is public under the Risk Averse banner as an example of AI work that measures itself rather than claiming.

PART 2 · PHASE 10 RESULT (README SECTION 22)

Unknown-position fillers under a strict never-repeat rule: null

The decoder was allowed to call any rune a filler and was run over every key the book supplies, three sign conventions, both F variants, all 55 pages. Nothing met the pre-registered bar. Pushed in commits 3fea7a4 and f58489d.

Measure	Value
Tests	2,514,930
Largest z on a real page	5.64 (LP2-14, self-running key), gibberish; chance ceiling about 5.4
Cells meeting the pre-registered rule	0
Planted controls recovered	15 of 18 at z ≥ 6 and rank 1
Wall time	48 minutes on four shards

How the prior was set

Five null priors were tried on the planted controls before any real page was scored. The Bayesian value implied by the 22% rate recovered 14 of 18; a looser prior at −0.8 recovered 15; stiffer priors did worse because random keystreams lose less freedom than the true key does. The sweep ran at −0.8. The three misses are stated as limits: one prime-stream Beaufort plant at 30% nulls (z 3.1), one four-word phrase key (z 5.2), and one plant that ranked third behind two equivalent key offsets.

What was claimed on 27 October, before review

Section 21 had left two mechanisms that fit every ciphertext statistic. One, the soft re-draw over an independent key, was never breakable. The other, a strict never-repeat rule with fillers at unknown positions, was breakable only if the key under the fillers came from the book. The write-up said that branch was now excluded for every book-derived family, and that the unsolved pages are consistent only with key material that was never published. Not covered, and said so: non-uniform fillers, fillers that consume a key value, rates outside 0.15 to 0.30, and key streams outside the repository's library.

PART 3 · WHAT AN EXTERNAL REVIEW CHANGED

Corrections adopted after an independent review of commit f58489d

A reviewer reproduced the central numbers (checksums, 12,956 runes, 86 doublets against 445 expected, the 0.0364 rate) and found five issues. Checked against the code, all five hold. The data facts stand; the wording of the conclusions does not.

Finding	Checked	What it changes
Threshold z ≥ 6 misses correct keys on short pages (1 of 20 at 66 runes, 14 of 20 at 92, 20 of 20 at 240).	Holds. 9 of 55 pages are under 200 runes, 3 under 100. The shortest Phase 10 control page (159 runes) was the z 3.1 miss.	Exclusions on short pages are inconclusive, not negative. Power must be reported by page length and family.

The closing sentence ("only unpublished key material remains") does not follow from the searches.	Holds. Untested texts, encodings, start states and mechanisms remain open; two planted controls scored below the cutoff.	Replace with: no accepted decoding within the tested configurations, with the stated detection limits.
The decoder allows at most two consecutive key rejections; the hard rule can require more.	Holds (score.py, rejection "one" takes skips of 1 or 2 only). Rare for random-like keys, more likely for keys with runs of repeated symbols.	Add a bounded repeated-rejection transition and an encoder/decoder consistency test on long identical runs.
Reuse detector: null moments clamp above length 250; "a quarter of pages gives a quarter of the pooled signal" is wrong (pairs scale with the square: 78 of 1,485 pairs, about 5%).	Holds. Pages run to 277 runes. A 13-page shared cluster would give about 7 sigma pooled, not 34; clusters under about 8 pages were never detectable by the pooled test.	Correct README lines 82, 120 and 268 and the diff_score note; extend the null table past 250; state the small-cluster gap.
"Independent key gives repeats at 1/29 whatever it does" assumes a uniform key, not independence alone.	Holds. A constant key is independent of the plaintext and preserves its doublets.	Wording: the simulated constructions are rejected, not every independent-key cipher.

Also noted: Phase 10 ran at NULLPEN −0.8 while the code default is −1.27. The run value is in the results file and the module header but should be stored in each result's JSON and made the default.

The revised position

What survives: the fivefold doublet shortage is real and homogeneous across pages; the two simulated mechanisms the Quorum panel called equivalent are distinguishable; every book-derived key family tested is excluded on pages long enough for the detector to see a plant. What does not survive: the claim that the branch is closed on every page, and the claim that only unpublished key material remains. The next priority, before any further key search, is to measure the detector's power by page length and family, lift the two-rejection bound, and recalibrate the reuse detector.

Prepared from README sections 1 to 22, results/phase10.md, results/verdict_check.md and the external review of 27 October 2026. Claims marked "holds" were re-checked against src/score.py, src/diff_score.py, src/null_lane.py and the page-length table.